

CAHIER D'ANALYSE

DAMLEGEND — Mars 2026

N°	Section	Page
I	Modules et Fonctionnalités	2
II	Acteurs / Utilisateurs	3
III	Objets du Système	4
IV	Diagrammes de Séquences	8
V	Règles de Gestion	14

I. Modules et Fonctionnalités

Le tableau suivant recense les modules de l'application et les fonctionnalités associées. Certains modules (ex. Tableau de bord) sont des vues synthétiques sans action propre : ils agrègent des données mais ne constituent pas des fonctionnalités à part entière.

Module	Fonctionnalité(s)	Type	Description
Authentification	Connexion via Wallet	Fonctionnalité	Connexion sécurisée à l'application via MetaMask
	Déconnexion	Fonctionnalité	Fermeture de la session utilisateur
Tableau de bord	Vue d'ensemble	Vue / Agrégat	Affichage consolidé : statistiques, campagnes actives, solde wallet
Campagnes	Créer une campagne	Fonctionnalité	Saisie et enregistrement d'une nouvelle campagne de financement
	Consulter les campagnes	Fonctionnalité	Liste paginée de toutes les campagnes disponibles
	Voir le détail d'une campagne	Fonctionnalité	Affichage complet : objectif, montant collecté, échéance, contributeurs
	Modifier une campagne	Fonctionnalité	Modification des informations par le créateur (avant tout financement)
	Clôturer / Annuler une campagne	Fonctionnalité	Clôture anticipée par le créateur ou expiration automatique
Contributions	Financer un projet	Fonctionnalité	Envoi de fonds depuis le wallet vers le smart contract
	Historique des contributions	Vue / Agrégat	Liste des transactions effectuées par l'utilisateur
Gestion des fonds	Retrait des fonds	Fonctionnalité	Retrait par le créateur si l'objectif est atteint en fin de campagne
	Remboursement automatique	Fonctionnalité	Remboursement automatique des contributeurs si l'objectif n'est pas atteint
Espace Utilisateur	Mon profil	Vue / Agrégat	Affichage et modification des informations personnelles
	Mes campagnes	Vue / Agrégat	Liste des campagnes créées par l'utilisateur connecté
	Mes contributions	Vue / Agrégat	Historique des financements réalisés

Module	Fonctionnalité(s)	Type	Description
	Gestion des utilisateurs	Fonctionnalité	Activation / désactivation / suppression logique des comptes
Administration	Gestion des campagnes (admin)	Fonctionnalité	Modération : validation, suspension, suppression logique
	Tableau de bord admin	Vue / Agrégat	Vue globale : KPIs, transactions, utilisateurs, alertes

Légende — **Fonctionnalité** : action métier réelle avec interaction système / **Vue / Agrégat** : module de consultation sans traitement propre.

II. Acteurs / Utilisateurs

Les acteurs représentent les intervenants humains qui interagissent avec l'application. Pour chaque acteur, le tableau précise les modules auxquels il a accès et son rôle.

Acteur	Rôle	Modules accessibles	Droits
Visiteur (non connecté)	Personne consultant l'application sans wallet connecté	• Consultation des campagnes • Détail d'une campagne	Lecture seule
Contributeur (connecté)	Utilisateur wallet connecté qui finance des projets	• Tableau de bord • Campagnes (consultation + détail) • Financement d'un projet • Historique contributions • Remboursement • Espace utilisateur	Lecture + Contribution
Créateur (connecté)	Utilisateur wallet connecté qui crée et gère ses campagnes	• Tout ce que le Contributeur peut faire • Créer / Modifier / Clôturer campagne • Retrait des fonds • Mes campagnes	Lecture + Contribution + Gestion campagne
Administrateur	Gestionnaire de la plateforme avec accès complet	• Tous les modules • Gestion utilisateurs • Modération campagnes • Tableau de bord admin • Paramètres système	Accès total

Hiérarchie des droits d'accès

Visiteur	Contributeur	Créateur	Administrateur
Consultation des campagnes	+ Financement + Espace perso	+ Créer/Gérer ses campagnes + Retrait fonds	+ Gestion utilisateurs + Modération + Paramètres

III. Objets du Système

Un objet est un élément abstrait du système disposant d'attributs et représentant un ensemble d'entités manipulées par le système. Chaque objet correspond à une table de la base de données relationnelle PostgreSQL. Conformément aux consignes, tous les objets partagent les attributs communs suivants :

Attribut commun	Type	Description
id	INTEGER (PK, AUTO_INCREMENT)	Clé primaire auto-incrémentée — identifiant unique de chaque enregistrement
is_visible	BOOLEAN (défaut : TRUE)	Suppression logique — FALSE = enregistrement masqué, accessible uniquement aux utilisateurs autorisés
is_active	BOOLEAN (défaut : TRUE)	Désactivation — FALSE = entité inactive (ex. utilisateur ne peut pas se connecter, campagne non modifiable)
created_by	INTEGER (FK → Utilisateur)	Identifiant de l'utilisateur ayant créé l'enregistrement — traçabilité des créations
created_at	TIMESTAMP	Date et heure de création de l'enregistrement
updated_at	TIMESTAMP	Date et heure de la dernière modification

NB : Ces attributs communs ne sont pas répétés dans les tableaux d'objets ci-dessous pour éviter la redondance.

3.1 Utilisateur (users)

Représente tout individu inscrit sur la plateforme. Un utilisateur possède un wallet crypto servant d'identifiant blockchain et un profil applicatif stocké en PostgreSQL.

Attribut	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant unique (commun)
wallet_address	VARCHAR(42)	UNIQUE NOT NULL	Adresse Ethereum du portefeuille MetaMask
username	VARCHAR(100)	NOT NULL	Pseudonyme de l'utilisateur
email	VARCHAR(255)	UNIQUE	Adresse email (optionnelle)
avatar_url	TEXT	NULLABLE	URL de l'image de profil
role	ENUM	NOT NULL	'visitor' 'contributor' 'creator' 'admin'
bio	TEXT	NULLABLE	Description courte du profil
last_login_at	TIMESTAMP	NULLABLE	Date de la dernière connexion
is_visible	BOOLEAN	DEFAULT TRUE	Suppression logique (commun)

Attribut	Type	Contrainte	Description
is_active	BOOLEAN	DEFAULT TRUE	Désactivation — connexion impossible si FALSE (commun)
created_by	INTEGER	FK → users.id	Admin ayant créé le compte (commun)

3.2 Campagne (campaigns)

Représente un projet de financement participatif créé par un utilisateur. Les métadonnées sont stockées en PostgreSQL ; la gestion des fonds est assurée par le smart contract.

Attribut	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant unique
title	VARCHAR(255)	NOT NULL	Titre de la campagne
description	TEXT	NOT NULL	Description détaillée du projet
goal_amount	DECIMAL(18,8)	NOT NULL > 0	Objectif de financement (en ETH ou token)
current_amount	DECIMAL(18,8)	DEFAULT 0	Montant collecté à ce jour
deadline	TIMESTAMP	NOT NULL	Date limite de la campagne
status	ENUM	NOT NULL	'draft' 'active' 'succeeded' 'failed' 'canceled'
category	VARCHAR(100)	NULLABLE	Catégorie du projet (tech, art, social...)
image_url	TEXT	NULLABLE	Image principale de la campagne
contract_address	VARCHAR(42)	NULLABLE	Adresse du smart contract déployé sur Ethereum
creator_id	INTEGER	FK → users.id NOT NULL	Créateur de la campagne
is_visible	BOOLEAN	DEFAULT TRUE	Suppression logique (commun)
is_active	BOOLEAN	DEFAULT TRUE	Désactivation — campagne non modifiable si FALSE (commun)
created_by	INTEGER	FK → users.id	Identité du créateur (commun)

3.3 Contribution (contributions)

Enregistre chaque acte de financement effectué par un contributeur vers une campagne. La transaction blockchain est référencée par son hash pour assurer la traçabilité on-chain.

Attribut	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant unique
campaign_id	INTEGER	FK → campaigns.id NOT NULL	Campagne financée
contributor_id	INTEGER	FK → users.id NOT NULL	Utilisateur contributeur
amount	DECIMAL(18,8)	NOT NULL > 0	Montant de la contribution (ETH)
tx_hash	VARCHAR(66)	UNIQUE NOT NULL	Hash de la transaction Ethereum
status	ENUM	NOT NULL	'pending' 'confirmed' 'refunded' 'failed'
block_number	BIGINT	NULLABLE	Numéro de bloc de confirmation
contributed_at	TIMESTAMP	NOT NULL	Horodatage de la contribution
is_visible	BOOLEAN	DEFAULT TRUE	Suppression logique (commun)
is_active	BOOLEAN	DEFAULT TRUE	Désactivation (commun)
created_by	INTEGER	FK → users.id	Traçabilité (commun)

3.4 Transaction Blockchain (blockchain_transactions)

Journal de toutes les transactions blockchain liées à l'application (contributions, retraits, remboursements, déploiements de contrats).

Attribut	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant unique
tx_hash	VARCHAR(66)	UNIQUE NOT NULL	Hash unique de la transaction Ethereum
type	ENUM	NOT NULL	'contribution' 'withdrawal' 'refund' 'deploy',
from_address	VARCHAR(42)	NOT NULL	Adresse source (wallet expéditeur)
to_address	VARCHAR(42)	NOT NULL	Adresse destination (contrat ou wallet)
amount	DECIMAL(18,8)	DEFAULT 0	Montant transféré en ETH
gas_used	BIGINT	NULLABLE	Gas consommé par la transaction
block_number	BIGINT	NULLABLE	Numéro du bloc Ethereum

Attribut	Type	Contrainte	Description
status	ENUM	NOT NULL	'pending' 'success' 'failed'
campaign_id	INTEGER	FK → campaigns.id NULLABLE	Campagne concernée (si applicable)
user_id	INTEGER	FK → users.id NULLABLE	Utilisateur concerné
is_visible	BOOLEAN	DEFAULT TRUE	Suppression logique (commun)
is_active	BOOLEAN	DEFAULT TRUE	Désactivation (commun)
created_by	INTEGER	FK → users.id	Traçabilité (commun)

3.5 Notification (notifications)

Gère les alertes et notifications envoyées aux utilisateurs : contribution reçue, objectif atteint, remboursement disponible, etc.

Attribut	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant unique
user_id	INTEGER	FK → users.id NOT NULL	Destinataire de la notification
type	ENUM	NOT NULL	'contribution' 'goal_reached' 'refund' 'campaign_end' 'system'
title	VARCHAR(255)	NOT NULL	Titre court de la notification
message	TEXT	NOT NULL	Corps du message
is_read	BOOLEAN	DEFAULT FALSE	Statut de lecture
link	TEXT	NULLABLE	URL de redirection associée
is_visible	BOOLEAN	DEFAULT TRUE	Suppression logique (commun)
is_active	BOOLEAN	DEFAULT TRUE	Désactivation (commun)
created_by	INTEGER	FK → users.id	Système ou admin ayant créé la notif (commun)

3.6 Catégorie (categories)

Référentiel des catégories de campagnes permettant le filtrage et la classification des projets.

Attribut	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant unique
name	VARCHAR(100)	UNIQUE NOT NULL	Nom de la catégorie (ex. Technologie, Art)
slug	VARCHAR(100)	UNIQUE NOT NULL	Identifiant URL-friendly (ex. technologie)
description	TEXT	NULLABLE	Description de la catégorie
icon_url	TEXT	NULLABLE	Icône représentative
is_visible	BOOLEAN	DEFAULT TRUE	Suppression logique (commun)
is_active	BOOLEAN	DEFAULT TRUE	Désactivation — catégorie masquée si FALSE (commun)
created_by	INTEGER	FK → users.id	Admin ayant créé la catégorie (commun)

IV. Diagrammes de Séquences / Description Textuelle

Cette section détaille, pour chaque fonctionnalité clé, les interactions entre les acteurs et le système selon un ordre chronologique strict. Chaque fonctionnalité est décrite textuellement puis illustrée par le diagramme de séquence correspondant.

4.1 Connexion via Wallet (MetaMask)

Étape	Acteur	Action / Message	Réponse Système
1	Utilisateur	Clique sur « Connecter Wallet »	Le frontend détecte la présence de MetaMask dans le navigateur
2	Frontend	Appelle <code>window.ethereum.request({ method: 'eth_requestAccounts' })</code>	MetaMask affiche une fenêtre de demande d'autorisation
3	MetaMask	L'utilisateur approuve la connexion	MetaMask retourne l'adresse wallet (ex. 0xABC...)
4	Frontend	Envoie l'adresse wallet au backend (POST <code>/api/auth/connect</code>)	Backend vérifie si l'adresse existe en PostgreSQL
5	Backend/DB	Requête SELECT sur la table users WHERE <code>wallet_address = ?</code>	Si nouveau : création du profil utilisateur. Si existant : mise à jour <code>last_login_at</code>
6	Backend	Génère un JWT signé et le renvoie au frontend	Frontend stocke le token et redirige vers le tableau de bord
7	Système	Vérification <code>is_active = TRUE</code>	Si FALSE → message d'erreur « Compte désactivé », connexion refusée

4.2 Création d'une Campagne

Étape	Acteur	Action / Message	Réponse Système
1	Créateur	Remplit le formulaire de création (titre, description, objectif, deadline, catégorie, image)	Le frontend valide les champs côté client (champs obligatoires, format dates, montant > 0)
2	Frontend	Soumet les données via POST <code>/api/campaigns</code>	Backend reçoit les données et vérifie l'authentification JWT
3	Backend	Valide les règles métier (deadline > date du jour, objectif > 0)	Si invalide : retourne 400 Bad Request avec message d'erreur
4	Backend/DB	INSERT dans campaigns (status = 'draft', <code>is_visible = TRUE</code> , <code>is_active = TRUE</code>)	PostgreSQL confirme l'enregistrement et retourne l'ID généré
5	Backend	Retourne <code>campaign_id</code> au frontend	Frontend affiche la page de la campagne créée

Étape	Acteur	Action / Message	Réponse Système
6	Créateur	(Optionnel) Lance le déploiement du smart contract via MetaMask	Le smart contract est déployé sur Sepolia, le contract_address est mis à jour en DB
7	Système	Notification envoyée au créateur (INSERT notifications)	Campagne visible dans la liste publique après validation admin

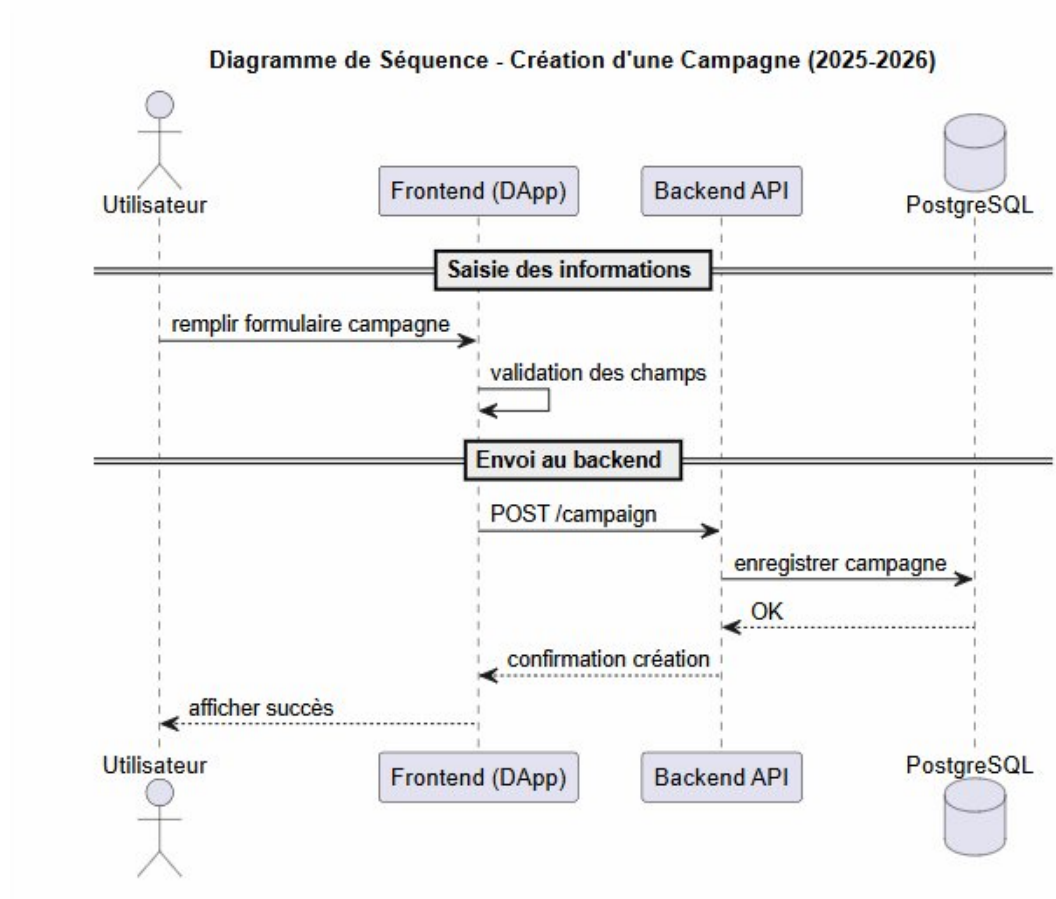


Figure 4.2 — Diagramme de séquence : Création d'une Campagne

4.3 Financement d'un Projet (Contribution)

Étape	Acteur	Action / Message	Réponse Système
1	Contributeur	Consulte la page d'une campagne active et clique sur « Financer »	Frontend affiche le formulaire de contribution avec le solde wallet actuel
2	Contributeur	Saisit le montant en ETH et confirme	Frontend appelle le smart contract via ethers.js : contribute()

Étape	Acteur	Action / Message	Réponse Système
3	MetaMask	Affiche la transaction à signer (montant + gas estimé)	L'utilisateur signe et envoie la transaction
4	Blockchain	Transaction diffusée sur le réseau Sepolia	Attente de confirmation du bloc (pending → confirmed)
5	Backend	Écoute les événements du smart contract (ContributionReceived)	À la réception : INSERT dans contributions (status='confirmed') et UPDATE campaigns.current_amount
6	Backend/DB	Vérifie si current_amount >= goal_amount	Si oui : UPDATE campaigns.status = 'succeeded' + INSERT notification au créateur
7	Système	INSERT notification au contributeur (confirmation de contribution)	Frontend affiche confirmation avec tx_hash et lien vers Etherscan

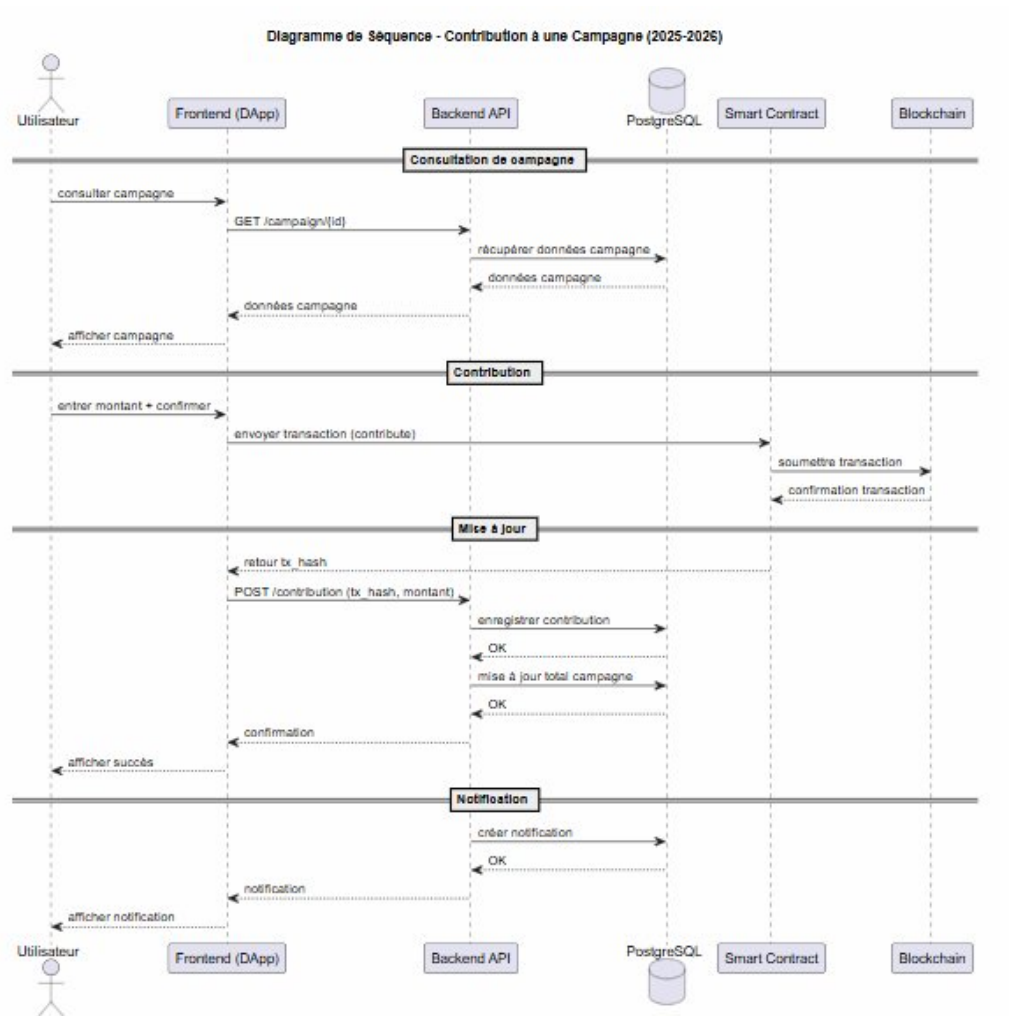


Figure 4.3 — Diagramme de séquence : Contribution à une Campagne

4.4 Retrait des Fonds (par le Créateur)

Étape	Acteur	Action / Message	Réponse Système
1	Créateur	Accède à sa campagne (statut 'succeeded') et clique sur « Retirer les fonds »	Frontend vérifie le statut de la campagne et la deadline dépassée
2	Frontend	Appelle la fonction withdraw() du smart contract via ethers.js	MetaMask affiche la transaction de retrait à signer
3	MetaMask	Créateur signe la transaction	Transaction envoyée sur la blockchain
4	Smart Contract	Vérifie : goal_amount atteint + deadline dépassée + sender = owner	Si toutes les conditions sont remplies : transfert ETH vers le wallet du créateur
5	Blockchain	Événement FundsWithdrawn émis	Backend écoute l'événement et met à jour campaigns.status = 'closed' en DB
6	Backend/DB	INSERT transaction (type='withdrawal') + INSERT notification créateur	Confirmation affichée sur l'interface

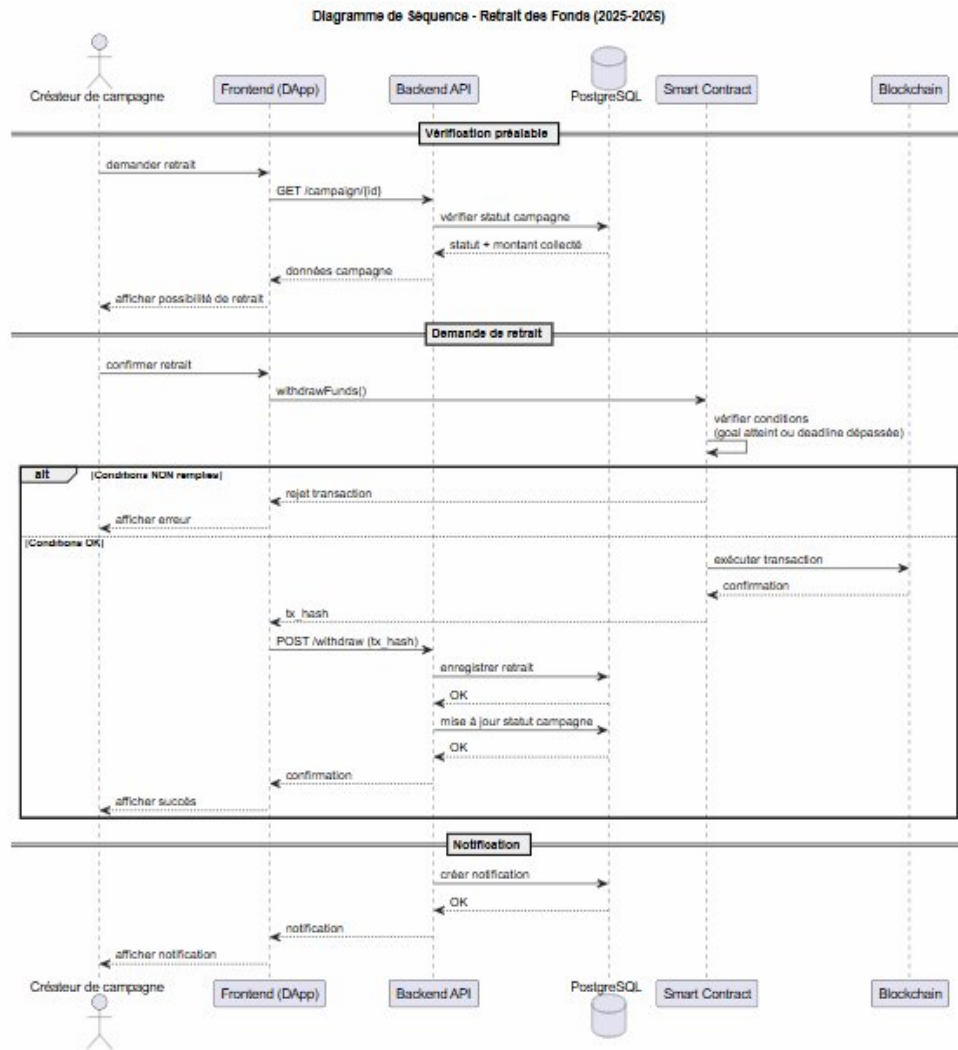


Figure 4.4 — Diagramme de séquence : Retrait des Fonds

4.5 Remboursement Automatique

Étape	Acteur	Action / Message	Réponse Système
1	Système (CRON)	Job planifié vérifie les campagnes dont deadline < NOW() et status = 'active'	Requête PostgreSQL : SELECT * FROM campaigns WHERE deadline < NOW() AND status = 'active'
2	Backend	Pour chaque campagne expirée : vérifie current_amount < goal_amount	Si objectif non atteint : déclenche le processus de remboursement
3	Smart Contract	Appel de la fonction refundAll() sur le contrat	Le contrat itère sur tous les contributeurs et retourne les fonds
4	Blockchain	Transactions de remboursement émises sur la blockchain	Événement RefundIssued émis pour chaque contributeur

Étape	Acteur	Action / Message	Réponse Système
5	Backend	Écoute les événements RefundIssued	UPDATE contributions.status = 'refunded' pour chaque tx + UPDATE campaigns.status = 'failed'
6	Backend/DB	INSERT notifications pour chaque contributeur remboursé	UPDATE campaigns.is_active = FALSE
7	Contributeur	Reçoit la notification de remboursement + les fonds dans son wallet	Frontend affiche le statut 'Remboursé' dans l'historique

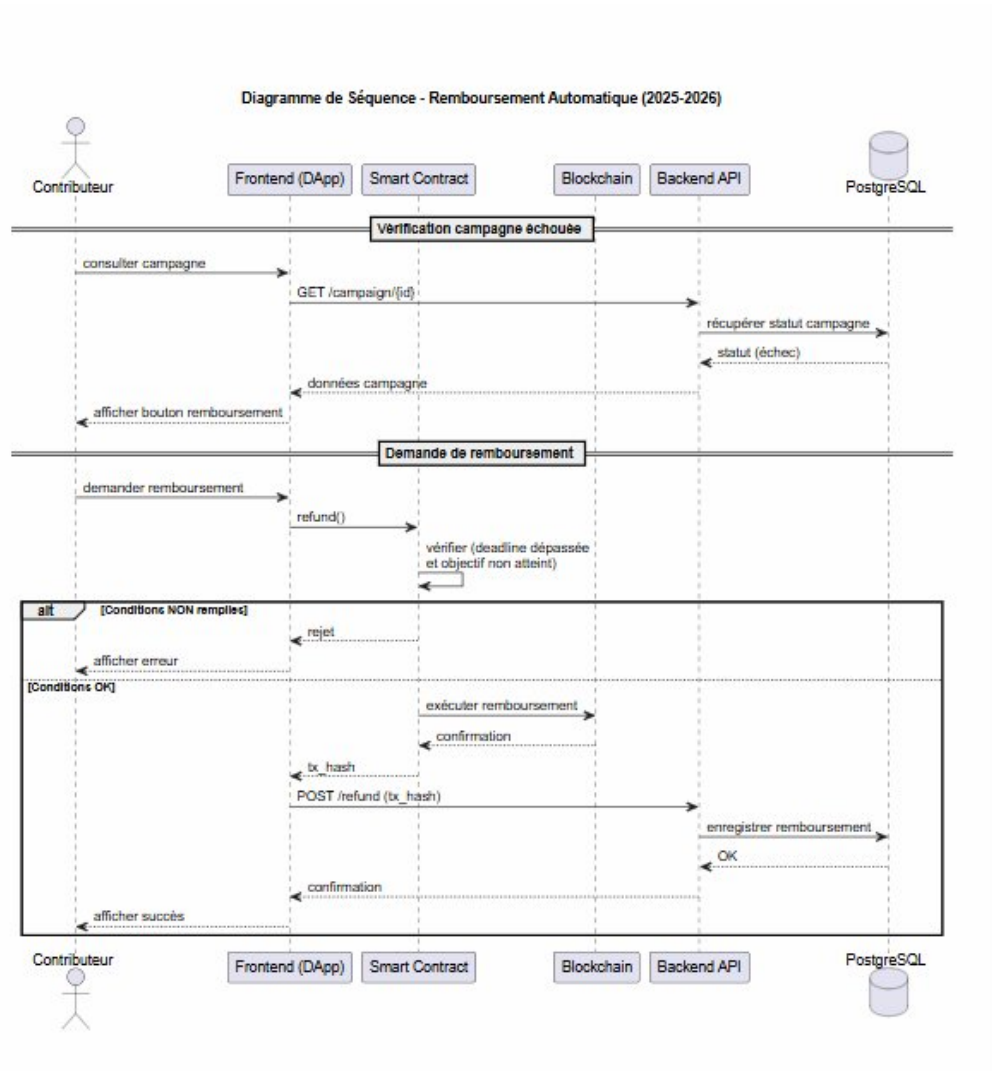


Figure 4.5 — Diagramme de séquence : Remboursement Automatique

4.6 Gestion Administrative (Désactivation / Suppression Logique)

Étape	Acteur	Action / Message	Réponse Système
1	Administrateur	Accède au tableau de bord admin > Gestion utilisateurs ou Campagnes	Backend vérifie le rôle = 'admin' via JWT
2	Admin	Sélectionne un élément et choisit Désactiver ou Supprimer (logique)	Frontend affiche une boîte de confirmation
3	Admin	Confirme l'action	Frontend envoie PATCH /api/admin/users/:id ou /api/admin/campaigns/:id
4	Backend	Désactivation : UPDATE SET is_active = FALSE Suppression logique : UPDATE SET is_visible = FALSE	PostgreSQL confirme la mise à jour
5	Système	Si utilisateur désactivé (is_active=FALSE) : invalidation du JWT actif	Prochaine connexion refusée avec message 'Compte désactivé'
6	Backend/DB	INSERT dans audit_logs (action, target_id, admin_id, timestamp)	Tableau de bord admin mis à jour en temps réel

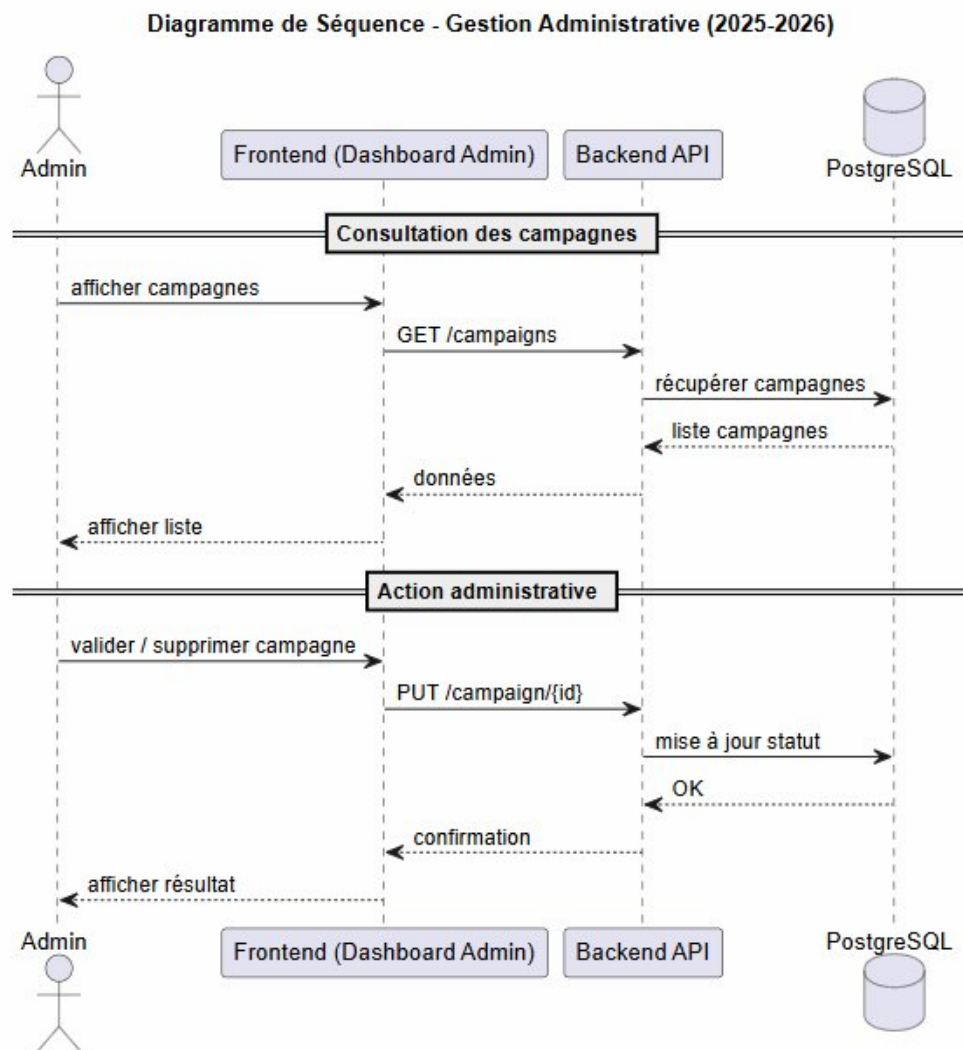


Figure 4.6 — Diagramme de séquence : Gestion Administrative

Diagramme de Classes UML

Le diagramme ci-dessous représente les classes du système, leurs attributs et les relations entre elles. Il constitue la base de la conception de la base de données relationnelle PostgreSQL.

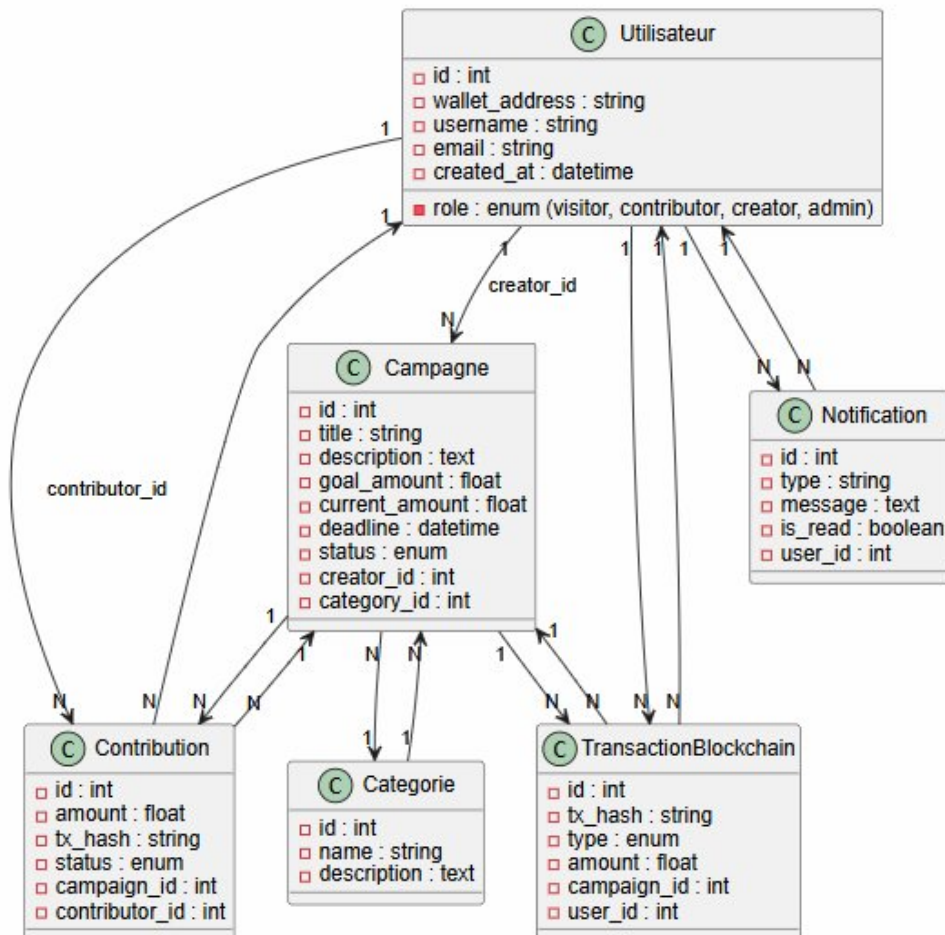


Figure — Diagramme de classes UML : DApp Crowdfunding

V. Règles de Gestion

Les règles de gestion définissent les contraintes et relations entre les objets du système. Elles constituent la base du diagramme de classes et guident la conception de la base de données.

5.1 Règles relatives aux Utilisateurs

Réf.	Règle de gestion	Condition / Contrainte
RG-U01	Un utilisateur est identifié de manière unique par son adresse wallet Ethereum	wallet_address UNIQUE NOT NULL, format 0x + 40 hex chars
RG-U02	Un utilisateur possède un et un seul rôle parmi : visitor, contributor, creator, admin	ENUM, NOT NULL, valeur par défaut : 'contributor'
RG-U03	Un utilisateur dont is_active = FALSE ne peut pas se connecter à l'application	Vérification obligatoire à chaque tentative de connexion
RG-U04	Un utilisateur dont is_visible = FALSE n'est pas accessible aux rôles non-admin	Filtrage WHERE is_visible = TRUE par défaut sur toutes les requêtes publiques
RG-U05	Seul un administrateur peut créer, modifier le rôle ou supprimer logiquement un utilisateur	Contrôle d'accès basé sur le rôle (RBAC) vérifié côté backend
RG-U06	Un utilisateur peut avoir plusieurs campagnes créées (relation 1-N avec campaigns)	campaigns.creator_id FK → users.id
RG-U07	Un utilisateur peut effectuer plusieurs contributions (relation 1-N avec contributions)	contributions.contributor_id FK → users.id

5.2 Règles relatives aux Campagnes

Réf.	Règle de gestion	Condition / Contrainte
RG-C01	Une campagne appartient à un et un seul créateur	campaigns.creator_id NOT NULL, FK → users.id
RG-C02	L'objectif de financement (goal_amount) doit être strictement supérieur à zéro	CHECK (goal_amount > 0)
RG-C03	La deadline doit être postérieure à la date de création	CHECK (deadline > created_at)
RG-C04	Une campagne ne peut être modifiée que si is_active = TRUE et status IN ('draft', 'active')	Vérification métier côté backend avant tout UPDATE
RG-C05	Le statut d'une campagne suit une machine à états : draft → active → succeeded/failed/cancelled	Les transitions invalides sont rejetées (ex. failed → active impossible)
RG-C06	Une campagne avec is_visible = FALSE n'est pas affichée dans les listes publiques	Filtre WHERE is_visible = TRUE sur toutes les requêtes publiques

Réf.	Règle de gestion	Condition / Contrainte
RG-C07	Une campagne peut recevoir plusieurs contributions (relation 1-N avec contributions)	contributions.campaign_id FK → campaigns.id
RG-C08	Une campagne peut appartenir à une catégorie (relation N-1 avec categories)	campaigns.category_id FK → categories.id NULLABLE

5.3 Règles relatives aux Contributions

Réf.	Règle de gestion	Condition / Contrainte
RG-K01	Une contribution ne peut être effectuée que vers une campagne active (status = 'active')	Vérification du statut avant d'appeler le smart contract
RG-K02	Le montant d'une contribution doit être strictement supérieur à zéro	CHECK (amount > 0) côté DB et validation côté smart contract
RG-K03	Chaque contribution est liée à une transaction blockchain unique (tx_hash)	tx_hash UNIQUE NOT NULL
RG-K04	Un contributeur ne peut pas financer sa propre campagne	CHECK (contributor_id != campaigns.creator_id) vérifié côté backend
RG-K05	Le remboursement n'est déclenché que si la campagne expire avec current_amount < goal_amount	Condition : deadline < NOW() AND status = 'active' AND current_amount < goal_amount
RG-K06	Une contribution remboursée passe au statut 'refunded' et ne peut plus être modifiée	is_active = FALSE après remboursement

5.4 Règles relatives aux Fonds et au Smart Contract

Réf.	Règle de gestion	Condition / Contrainte
RG-F01	Le retrait des fonds n'est autorisé que si goal_amount est atteint ET la deadline est dépassée	Vérification onchain : require(current_amount >= goal_amount && block.timestamp > deadline)
RG-F02	Seul le créateur de la campagne peut effectuer le retrait des fonds	require(msg.sender == owner) dans le smart contract
RG-F03	Le retrait ne peut être effectué qu'une seule fois par campagne	Booléen withdrawn = true dans le smart contract après retrait
RG-F04	Les fonds sont bloqués dans le smart contract jusqu'à la fin de la campagne	Aucun transfert direct possible en dehors des fonctions contribute/withdraw/refund
RG-F05	En cas d'échec (objectif non atteint), le remboursement est automatique et total	Chaque contributeur récupère exactement le montant qu'il a versé

5.5 Règles relatives aux Attributs Communs (is_visible / is_active / created_by)

Réf.	Règle de gestion	Application
RG-G01	Tout objet avec is_visible = FALSE est traité comme supprimé logiquement	Filtrage automatique WHERE is_visible = TRUE sur toutes les requêtes non-admin
RG-G02	Seuls les utilisateurs avec le rôle 'admin' peuvent consulter les objets is_visible = FALSE	RBAC vérifié côté backend à chaque requête

Réf.	Règle de gestion	Application
RG-G03	Un objet avec is_active = FALSE ne peut faire l'objet d'aucune modification	Vérification obligatoire avant tout UPDATE : WHERE is_active = TRUE
RG-G04	L'attribut created_by est renseigné automatiquement à la création par le système	Valeur = ID de l'utilisateur authentifié au moment de la création
RG-G05	La suppression physique (DELETE) est interdite dans l'application pour tous les objets	Seule la suppression logique (is_visible = FALSE) est autorisée

5.6 Relations entre les Objets (Synthèse du Diagramme de Classes)

Objet Source	Card.	Objet Cible	Attribut de liaison	Description
Utilisateur	1	N Campagnes	campaigns.creator_id	Un créateur possède plusieurs campagnes
Campagne	1	N Contributions	contributions.campaign_id	Une campagne reçoit plusieurs contributions
Utilisateur	1	N Contributions	contributions.contributor_id	Un contributeur effectue plusieurs contributions
Campagne	N	1 Catégorie	campaigns.category_id	Plusieurs campagnes appartiennent à une catégorie
Utilisateur	1	N Notifications	notifications.user_id	Un utilisateur reçoit plusieurs notifications
Campagne	1	N Transactions	blockchain_transactions.campaign_id	Une campagne est liée à plusieurs transactions blockchain
Utilisateur	1	N Transactions	blockchain_transactions.user_id	Un utilisateur est à l'origine de plusieurs transactions